

Nghệ thuật giải bài (The Art of Problem Solving)

PHỤ THUỘC không — đây là chương mở đầu. Mọi chương sau đều giả định bạn đã có quy trình bốn bước và bộ heuristic của chương này trong đầu.

TÓM TẮT

Chương này dựng một quy trình để chạy khi bạn đứng trước một bài chưa biết làm: hiểu đề cho thật kỹ, tìm mối nối với thứ đã biết, thực hiện, rồi nhìn lại. Bốn bước đó là của Pólya, viết cho toán học năm 1945, và chúng áp thẳng vào lập trình mà gần như không phải sửa. Phần thứ hai của chương là bộ heuristic sinh ý tưởng — mười hai câu hỏi cụ thể để hỏi khi bí, mỗi câu kèm ví dụ thuật toán mà nó từng đẻ ra. Phần thứ ba là điều Schoenfeld phát hiện khi quay video người giải bài: cái phân biệt người giỏi với người kém không phải lượng kiến thức mà là *khả năng tự giám sát*. Nếu chỉ nhớ một điều: khi bí, đừng cố nghĩ mạnh hơn — hãy đổi câu hỏi bạn đang tự hỏi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán (problem)** — một tình huống trong đó bạn biết mình muốn gì nhưng chưa biết cách đi tới đó. Nếu bạn đã biết cách đi thì đó không còn là bài toán, chỉ là bài tập.
- **heuristic** — một mẹo tìm đường, không đảm bảo thành công nhưng làm tăng xác suất tìm ra ý tưởng. Khác với thuật toán ở chỗ nó không hứa hẹn gì cả.
- **vét cạn (brute force)** — cách giải duyệt qua mọi khả năng. Luôn đúng, thường quá chậm, nhưng là điểm xuất phát của gần như mọi lời giải nhanh.
- **bất biến (invariant)** — một mệnh đề luôn đúng tại một điểm nhất định trong thuật toán, chẳng hạn “sau mỗi vòng lặp, đoạn đầu của mảng đã sắp xếp”. Chương A/04 nói kỹ.
- **độ phức tạp thời gian** — cách nói “khi dữ liệu lớn gấp đôi thì thời gian chạy tăng bao nhiêu lần”. Chương A/03 nói kỹ.
- **ví dụ nhỏ nhất** — trường hợp nhỏ nhất mà vẫn còn giữ được cái khó của bài. Công cụ mạnh nhất và bị bỏ qua nhiều nhất trong chương này.

CÂU HỎI MỞ ĐẦU

- Q1. Hai người cùng biết đúng một lượng thuật toán như nhau, vì sao một người giải được bài mới còn người kia thì không?
- Q2. Vì sao lời khuyên “hãy nghĩ kỹ hơn” gần như vô dụng khi bạn đang bí, còn lời khuyên “hãy vẽ ví dụ $n = 4$ ra giấy” thì thường hiệu quả?
- Q3. Vét cạn là cách giải ai cũng nghĩ ra và luôn quá chậm. Vậy vì sao chương này lại bảo bạn phải viết nó ra trước?
- Q4. Bước “nhìn lại” của Pólya là bước bị bỏ qua nhiều nhất. Bỏ nó đi thì bạn mất chính xác cái gì?
- Q5. Khi nào thì việc bỏ hướng đang đi là quyết định đúng, và làm sao biết được điều đó *trước khi* hết giờ?

Hãy bắt đầu bằng một cảnh quen thuộc. Bạn đọc một đề bài, hiểu từng câu, và rồi không biết làm gì tiếp. Không phải bạn thiếu kiến thức — nếu ai đó nói “bài này dùng quy hoạch động” thì bạn làm được ngay. Cái thiếu nằm ở chỗ khác: bạn không có *quy trình* để đi từ đề bài tới cái gọi ý

đó. Trong lúc bí, đầu bạn chạy một vòng lặp vô ích — đọc lại đề, thấy vẫn thế, thử nhớ xem bài này giống bài nào từng gặp, không nhớ ra, đọc lại đề. Vòng lặp đó không hội tụ vì nó không thay đổi thông tin bạn có.

Điều đáng nói là tình trạng này đã được nghiên cứu nghiêm túc, và không phải bởi dân lập trình. Năm 1945, nhà toán học George Pólya viết *How to Solve It*[1], một quyển sách mỏng cố gắng trả lời câu hỏi “khi một người giỏi giải được một bài mới, trong đầu họ thực sự diễn ra chuyện gì”. Bốn mươi năm sau, Alan Schoenfeld làm một việc tàn nhẫn hơn: ông đặt camera quay lại quá trình giải toán của sinh viên và của các nhà toán học, rồi đối chiếu[2]. Kết quả của cả hai người, cộng với nghiên cứu về luyện tập có chủ đích của Ericsson[3], tạo thành nền cho chương này. Chúng ta sẽ dùng chúng theo nghĩa rất thực dụng: đây là quy trình để chạy khi không biết làm gì.

1.1 Bốn bước của Pólya, dịch sang ngôn ngữ lập trình

Pólya chia quá trình giải thành bốn bước: hiểu bài toán, lập kế hoạch, thực hiện kế hoạch, và nhìn lại. Nghe đơn giản đến mức dễ bị coi thường. Nhưng cái giá trị không nằm ở bốn cái tên mà nằm ở *những câu hỏi cụ thể* thuộc về mỗi bước, và ở việc bạn biết mình đang ở bước nào — vì phần lớn thất bại đến từ việc nhảy cóc sang bước 3 khi bước 1 còn chưa xong.

Bước 1 — Hiểu bài toán, kỹ hơn bạn tưởng là cần

Câu hỏi của bước này không phải “đề nói gì” mà là bốn câu rất cụ thể: *cái gì là dữ liệu vào, cái gì là kết quả cần ra, ràng buộc là gì, và tôi có thể phát biểu lại đề bằng lời của mình mà không nhìn vào đề không*. Câu cuối là bài kiểm tra thật sự. Nếu chưa phát biểu lại được, bạn chưa hiểu đề, và mọi phút suy nghĩ tiếp theo đều là phí.

Trong lập trình có ba việc thuộc bước này mà người mới hay bỏ qua. Thứ nhất, *đọc giới hạn*. Dòng “ $1 \leq n \leq 2 \cdot 10^5$ ” không phải chi tiết kỹ thuật, nó là một nửa gợi ý: nó nói lời giải cần cỡ $O(n \log n)$, tức là loại ngay mọi hướng $O(n^2)$. Chương A/03 có bảng chuyển từ giới hạn sang độ phức tạp cho phép; bảng đó nên thuộc lòng.

Thứ hai, *làm tay ví dụ nhỏ*. Không phải ví dụ trong đề — ví dụ của bạn, nhỏ hơn, và bạn tự tính đáp án. Với $n = 1, n = 2, n = 3$. Việc này nghe tầm thường nhưng nó là công cụ sinh ý tưởng mạnh nhất mà chương này có, vì cấu trúc của bài toán thường chỉ lộ ra khi bạn nhìn thấy các trường hợp cạnh nhau.

Thứ ba, *tìm trường hợp biên và trường hợp suy biến*. Mảng rỗng, tất cả phần tử bằng nhau, đồ thị không liên thông, số âm, tràn số. Danh sách này sẽ cứu bạn ở bước 4, nhưng nghĩ ra nó ở bước 1 thì rẻ hơn nhiều.

CẠM BẮY

Người mới thường coi bước 1 là bước “đọc đề”, mất hai phút, rồi lao vào nghĩ thuật toán. Người có kinh nghiệm dành *nhiều hơn* thời gian cho bước 1 chứ không ít hơn — vì đa số bài khó không khó ở thuật toán, mà khó ở chỗ bạn chưa nhìn ra bài toán thật sự đang hỏi gì. Một dấu hiệu chắc chắn rằng bạn bỏ dở bước 1: bạn đang viết code mà vẫn phải cuộn lên đọc lại đề.

Bước 2 – Lập kế hoạch: tìm mối nối

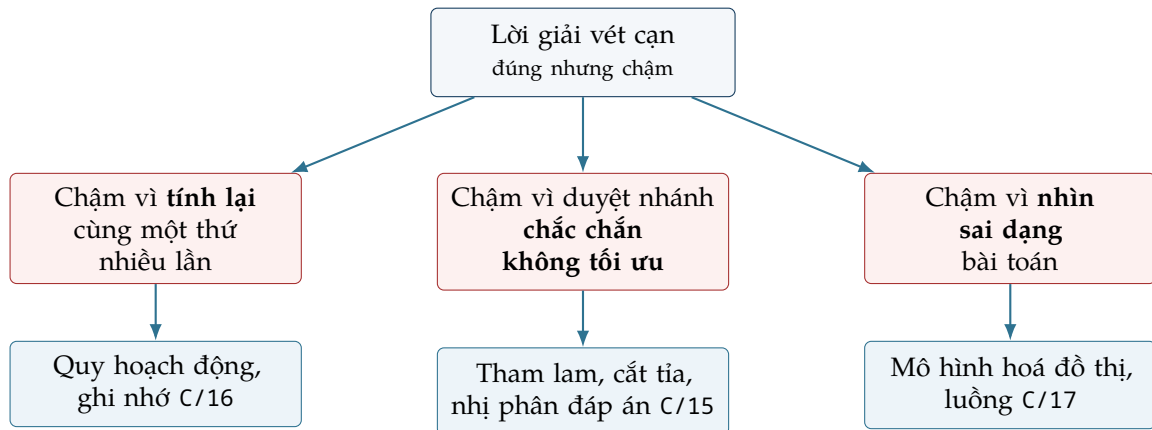
Đây là bước khó nhất và cũng là bước Pólya viết nhiều nhất. Câu hỏi trung tâm của ông rất đơn giản: *bạn đã gặp bài nào giống bài này chưa?* Không phải giống hệt — giống ở một khía cạnh nào đó. Cùng dạng dữ liệu vào, cùng dạng kết quả, cùng cấu trúc ràng buộc.

Trong lập trình, bước 2 có một hình thức rất cụ thể mà tôi sẽ nhắc lại nhiều lần trong quyển sách này: **luôn bắt đầu bằng vết cạn**. Viết ra — bằng lời hoặc bằng mã giả — một lời giải chắc chắn đúng dù chậm. Việc này có ba tác dụng, và cả ba đều lớn.

Tác dụng thứ nhất: nó buộc bạn phát biểu chính xác cái gì đang được tìm. Rất nhiều lúc bí là do bài toán trong đầu bạn vẫn còn mờ, và vết cạn làm nó rõ.

Tác dụng thứ hai: nó cho bạn một *lời giải đúng để đối chiếu*. Sau này, khi có lời giải nhanh, bạn cho hai chương trình chạy trên hàng nghìn test ngẫu nhiên nhỏ và so kết quả — kỹ thuật stress test ở chương E/25, và là kỹ thuật gỡ lỗi hiệu quả nhất trong nghề.

Tác dụng thứ ba, và là tác dụng quan trọng nhất về mặt tư duy: nó cho bạn một câu hỏi tốt hơn để suy nghĩ. Thay vì câu hỏi mù mờ “làm sao giải bài này”, bạn có câu hỏi sắc bén “*vì sao lời giải vết cạn này chậm, và công việc nào trong đó là thừa*”. Hình 1.1 cho thấy vì sao câu hỏi thứ hai dễ trả lời hơn hẳn: ba câu trả lời phổ biến dẫn thẳng tới ba nhóm kỹ thuật lớn của phần C.



Hình 1.1. Cây quyết định đi từ lời giải vết cạn tới nhóm kỹ thuật cần dùng. Câu hỏi “vì sao vết cạn chậm” có ít câu trả lời hơn hẳn câu hỏi “giải bài này thế nào”, nên nó dễ trả lời hơn — đó là toàn bộ giá trị của việc viết vết cạn ra trước.

Bước 3 – Thực hiện, và kiểm tra từng bước

Pólya nhấn mạnh một điều mà dân lập trình hay quên: khi thực hiện kế hoạch, phải kiểm tra từng bước có đúng không, và phải phân biệt được giữa “tôi thấy nó đúng” với “tôi chứng minh được nó đúng”. Trong lập trình, bước này gồm ba việc: chuyển ý tưởng thành mã, viết được các bất biến ra thành lời, và chạy thử trên ví dụ tự làm tay ở bước 1.

Có một quy tắc thực dụng đáng nhớ: *đừng gõ code khi bạn chưa nói được lời giải bằng ba câu*. Nếu bạn không tóm tắt được ý tưởng thành ba câu, tức là kế hoạch chưa xong, và gõ code lúc này chỉ chuyển sự mơ hồ từ đầu bạn vào trình soạn thảo — nơi nó khó sửa hơn nhiều.

Bước 4 – Nhìn lại: bước bị bỏ nhiều nhất, mất mát lớn nhất

Sau khi bài chạy đúng, gần như mọi người đóng bài và làm bài tiếp theo. Đây là lỗi tốn kém nhất trong toàn bộ việc luyện thuật toán, vì bước 4 chính là bước biến một lần giải bài thành một mẫu kiến thức dùng lại được.

Bước 4 gồm ba câu hỏi. Thứ nhất: *lời giải có thể ngắn hơn, nhanh hơn, hay đơn giản hơn không*. Thứ hai: *ý tưởng cốt lõi ở đây là gì, phát biểu bằng một câu, không nhắc tới chi tiết đề bài*. Thứ ba: *tôi đã bí ở đâu, và dấu hiệu nào lẽ ra đã giúp tôi thoát sớm hơn*. Câu thứ hai tạo ra thứ mà người luyện lâu năm gọi là “mẫu” — ví dụ “khi phải trả lời nhiều truy vấn tổng đoạn trên mảng tĩnh, hãy tính trước tổng tiền tố”. Câu thứ ba tạo ra nhật ký lỗi, công cụ được nói kỹ ở chương E/28.

CẦN NHỚ

Bốn bước không phải một đường thẳng mà là một vòng lặp có phản hồi. Bí ở bước 2 thì quay lại bước 1 (thường bạn hiểu sai đề); sai ở bước 3 thì quay lại bước 2 (thường kế hoạch có lỗ hổng, không phải code có lỗi gõ). Chỗ đáng ngờ nhất luôn là bước *ngay trước* chỗ bạn đang mắc.

1.2 Mười hai câu hỏi để hỏi khi bí

Phần này là công cụ thực dụng nhất của chương. Khi bí, đừng cố “nghĩ mạnh hơn” — đó là lời khuyên vô nghĩa vì nó không thay đổi thông tin bạn đang có. Thay vào đó, chạy qua danh sách dưới đây và tự hỏi từng câu. Mỗi câu hỏi có khả năng làm lộ ra một cấu trúc mà cách nhìn hiện tại của bạn đang che mắt. Danh sách này tổng hợp từ Pólya[1], Zeitz[4], Engel[5] và kinh nghiệm lập trình thi đấu.

1. **Làm ví dụ nhỏ nhất ra giấy.** $n = 1, 2, 3, 4$, tự tính đáp án bằng tay, xếp thành bảng, nhìn xem có quy luật gì. Đây là câu hỏi số một vì nó có tỉ lệ thành công cao nhất và chi phí thấp nhất.
2. **Bài này giống bài nào tôi từng giải?** Giống ở dữ liệu vào, ở kết quả ra, hay ở cấu trúc? Nếu giống một phần, phần khác biệt nằm ở đâu?
3. **Bỏ bớt một ràng buộc thì bài có dễ đi không?** Nếu mọi số đều dương thì sao? Nếu đồ thị là cây thì sao? Giải bài dễ hơn trước, rồi hỏi cách bù lại ràng buộc đã bỏ. Pólya gọi đây là “bài toán phụ”.
4. **Đảo ngược hướng nhìn.** Thay vì tìm cách tốt nhất, hãy đếm cách xấu và trừ đi. Thay vì đi từ đầu tới cuối, đi từ cuối về đầu. Thay vì hỏi “đáp án là bao nhiêu”, hỏi “với giá trị x cho trước, đáp án có $\geq x$ không” — câu này chính là cửa vào kỹ thuật nhị phân trên đáp án ở chương C/19.
5. **Sắp xếp lại dữ liệu thì sao?** Rất nhiều bài trở nên hiển nhiên sau khi sắp xếp, vì sắp xếp áp một thứ tự lên dữ liệu và thứ tự là thứ mà thuật toán tham lam cần.
6. **Có đại lượng nào không đổi không?** Tổng, chẵn lẻ, tổng theo modulo, số nghịch thế. Bất biến là công cụ mạnh nhất để chứng minh *không thể* làm được điều gì đó.
7. **Trường hợp cực đoan nói gì?** Cái nhỏ nhất, cái lớn nhất, cái ở biên. Lời giải tham lam thường sống ở đó.
8. **Có thể chia bài thành hai nửa độc lập không?** Nếu có thì chia để trị hoặc gặp nhau ở giữa. Nếu không, chỗ “đính” giữa hai nửa chính là trạng thái của quy hoạch động.
9. **Phát biểu lại bằng ngôn ngữ khác.** Bài mảng phát biểu lại thành bài đồ thị, bài đồ thị thành bài ma trận, bài đếm thành bài xác suất. Đổi ngôn ngữ hay làm hiện ra thuật toán có sẵn.
10. **Cái gì thực sự thay đổi giữa hai bước liên tiếp?** Nếu đáp án của bước $i + 1$ chỉ khác đáp án bước i một ít, bạn có lời giải tăng dần thay vì tính lại từ đầu — đó là ý tưởng nền của hai con trỏ, cửa sổ trượt và cấu trúc dữ liệu động.
11. **Nếu tôi được biết trước một đại lượng thì bài có dễ không?** Nếu “giá mà tôi biết phần tử lớn nhất nằm ở đâu” làm bài trở nên dễ, thì hãy duyệt qua mọi vị trí có thể của nó — kỹ thuật cố định một biến.

12. **Giới hạn đang mách gì?** $n \leq 20$ mách vét cạn 2^n ; $n \leq 10^5$ mách $O(n \log n)$; $n \leq 500$ mách $O(n^3)$; tổng n qua các test được chặn thì mách lời giải tuyến tính theo tổng. Chương A/03 có bảng đầy đủ.

TRỰC GIÁC

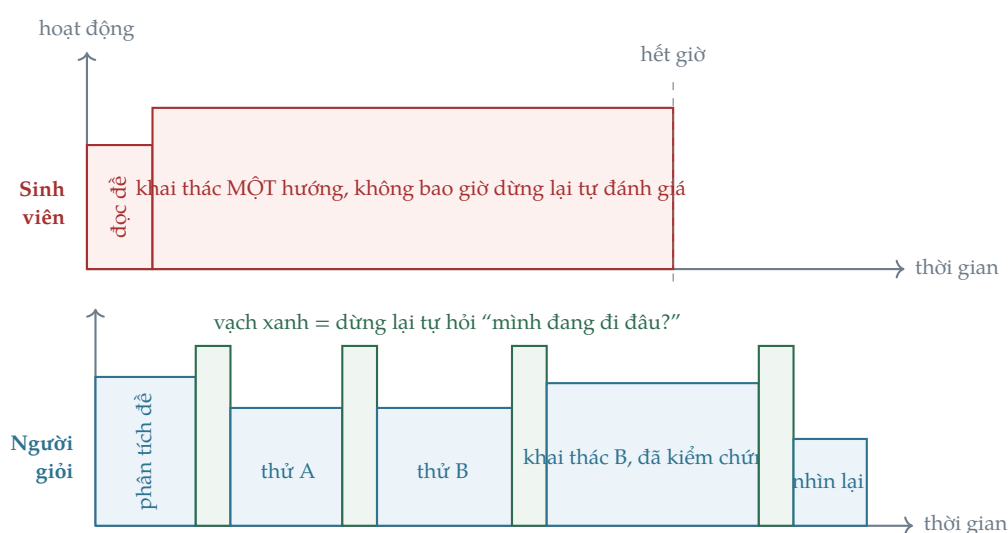
Vì sao một danh sách câu hỏi lại hiệu quả hơn “suy nghĩ tự do”? Vì bí không phải trạng thái thiếu năng lực tư duy, mà là trạng thái *lặp lại một cách nhìn*. Đầu bạn cứ chạy lại đúng biểu diễn cũ của bài toán và tất nhiên thu được đúng kết luận cũ. Mỗi câu hỏi trong danh sách buộc bạn thay đổi biểu diễn — vẽ ra, đảo chiều, bỏ ràng buộc, đổi ngôn ngữ. Ý tưởng mới gần như luôn đến từ biểu diễn mới, chứ không từ việc nhìn chăm chăm biểu diễn cũ lâu hơn.

1.3 Điều Schoenfeld quay được trên video

Đến đây có một câu hỏi đáng đặt: nếu bốn bước và mười hai câu hỏi đơn giản như vậy, vì sao không phải ai đọc xong cũng giải bài giỏi lên? Alan Schoenfeld có câu trả lời dựa trên dữ liệu, và nó thay đổi cách nhìn của tôi về việc luyện tập.

Ông quay video các nhóm sinh viên giải toán và so với video các nhà toán học chuyên nghiệp giải cùng loại bài[2]. Điểm khác biệt lớn nhất không nằm ở kiến thức, cũng không nằm ở việc ai biết nhiều heuristic hơn. Nó nằm ở *cách phân bổ thời gian*. Sinh viên điển hình đọc đề, chọn một hướng trong vòng vài phút, rồi đi theo hướng đó cho tới hết giờ — kể cả khi hướng đó không dẫn tới đâu. Nhà toán học thì liên tục dừng lại tự hỏi: “mình đang làm gì đây? cái này có dẫn tới đâu không? đã bao lâu rồi? có nên thử hướng khác không?”.

Schoenfeld gọi tầng đó là *control* — tạm dịch là tự giám sát. Ông tổng kết rằng năng lực giải bài gồm bốn thành phần: kiến thức nền, các heuristic, khả năng tự giám sát, và hệ niềm tin về việc giải toán là gì. Thành phần thứ ba là thứ tạo ra khác biệt lớn nhất giữa hai nhóm, và cũng là thành phần dễ luyện nhất — vì nó chỉ đòi hỏi bạn đặt đồng hồ và tự hỏi vài câu.



Hình 1.2. Hai kiểu phân bổ thời gian mà Schoenfeld quan sát được. Người giải kém dành gần như toàn bộ thời gian để khai thác hướng đầu tiên nghĩ ra. Người giải giỏi cắt quá trình bằng những lần dừng ngắn để tự đánh giá, nhờ đó bỏ được hướng sai sớm. Điểm mấu chốt: các vạch xanh gần như không tốn thời gian, nhưng chúng quyết định phần thời gian còn lại được dùng vào việc gì.

Bài học thực hành rất cụ thể. Khi luyện, hãy đặt một mốc thời gian — ví dụ 10 phút — và mỗi khi tới mốc thì dừng lại 20 giây để trả lời ba câu: *tôi đang thử hướng gì, hướng này đã cho tôi tiến bộ nào chưa, nếu 10 phút nữa vẫn thế thì tôi sẽ chuyển sang hướng nào*. Ba câu này nghe máy móc, nhưng chúng chính là thứ mà người giỏi làm tự động và người mới không làm bao giờ.

1.4 Một ví dụ đi hết bốn bước

Lý thuyết trên sẽ mờ nếu không có một ví dụ chạy hết. Hãy lấy một bài kinh điển, đủ nhỏ để làm tay và đủ giàu để lộ ra cách nghĩ: *cho một mảng số nguyên có cả âm lẫn dương, tìm đoạn con liên tiếp có tổng lớn nhất*. Đây là bài mà Jon Bentley kể lại trong *Programming Pearls*[6] như một ví dụ mẫu mực về việc một bài toán có thể được giải lại nhiều lần, mỗi lần tốt hơn.

Bước 1 — hiểu. Dữ liệu vào là mảng $a_1, \dots, a_n$, kết quả là số lớn nhất trong các tổng $a_i + a_{i+1} + \dots + a_j$. Phát biểu lại bằng lời của mình: “chọn một đoạn liên nhau sao cho tổng lớn nhất”. Trường hợp biên đáng ngờ ngay: nếu mọi phần tử đều âm thì sao? Tùy quy ước — hoặc lấy phần tử lớn nhất, hoặc cho phép đoạn rỗng với tổng 0. Đây chính là loại câu hỏi mà bước 1 phải làm rõ, vì nó quyết định code.

Bước 2 — kế hoạch. Vết cặn trước: duyệt mọi cặp (i, j) , tính tổng, lấy lớn nhất. Chi phí $O(n^3)$ nếu tính tổng lại từ đầu mỗi lần, $O(n^2)$ nếu cộng dồn. Bây giờ dùng câu hỏi trung tâm: *công việc nào đang bị lặp?* Rõ ràng khi ta cố định j và thử mọi i , phần lớn công sức lặp lại giữa j và $j + 1$. Đó là dấu hiệu của câu hỏi số 10 trong danh sách: *cái gì thay đổi giữa hai bước liên tiếp?*

Hãy gọi B_j là tổng lớn nhất của một đoạn kết thúc đúng tại j . Câu hỏi trở thành: biết B_j , tính B_{j+1} thế nào? Đoạn tốt nhất kết thúc tại $j + 1$ hoặc là chỉ gồm mình a_{j+1} , hoặc là đoạn tốt nhất kết thúc tại j nối thêm a_{j+1} . Vậy

$$B_{j+1} = \max(a_{j+1}, B_j + a_{j+1}) = a_{j+1} + \max(0, B_j),$$

và đáp án là $\max_j B_j$. Công thức nhỏ này nói một điều rất trực giác: *nếu phần đuôi tích lũy được cho tới giờ đang âm, hãy vứt nó đi và bắt đầu lại*. Chi phí xuống $O(n)$, bộ nhớ $O(1)$.

Bước 3 — thực hiện. Bất biến cần giữ khi viết vòng lặp: sau khi xử lý xong vị trí j , biến cur bằng B_j và biến best bằng $\max_{k \leq j} B_k$. Viết được bất biến ra thành lời như vậy thì code gần như tự viết, và mọi câu hỏi kiểu “khởi tạo bằng 0 hay bằng $-\infty$ ” đều có câu trả lời từ bất biến chứ không phải từ việc thử.

Bước 4 — nhìn lại. Ý tưởng cốt lõi, phát biểu không nhắc đề bài: *khi tìm cực trị trên mọi đoạn, hãy cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó*. Câu này là một mẫu dùng lại được — nó giải luôn các bài tích lớn nhất, đoạn có tổng chia hết cho k , và là bước đầu tiên của rất nhiều bài quy hoạch động một chiều. So sánh: nếu bạn chỉ nhớ “bài tổng đoạn lớn nhất dùng thuật toán Kadane” thì bạn có một cái tên; nếu bạn nhớ câu vừa rồi thì bạn có một công cụ.

CẦN NHỚ

Bước 4 là chỗ biến một lần giải bài thành một mẫu kiến thức. Cách làm cụ thể: viết ra một câu nói ý tưởng cốt lõi mà không được nhắc tới chi tiết đề bài. Nếu không viết được câu đó, bạn chưa thực sự hiểu vì sao lời giải chạy được.

1.5 Vòng lặp bí — thoát bằng cách nào

Còn một tình huống chưa nói: bạn đã chạy qua danh sách câu hỏi, vẫn bí. Lúc này có ba chiến lược, xếp theo thứ tự nên thử.

Thứ nhất, giải một bài dễ hơn. Không phải bỏ cuộc mà là đổi mục tiêu tạm thời: giải bài với n nhỏ, hoặc với một ràng buộc thêm vào cho dễ, hoặc chỉ giải một trường hợp riêng. Rất thường xuyên, lời giải của bài dễ hơn chứa đựng ý tưởng cần cho bài gốc.

Thứ hai, viết ra chính xác chỗ bạn kẹt. Một câu, dạng “tôi biết cách làm nếu như ... nhưng tôi không biết cách xử lý khi ...”. Việc viết ra buộc sự mơ hồ phải thành hình, và một nửa số lần thì chính lúc viết bạn nhìn ra hướng đi.

Thứ ba, rời bài ra. Đây không phải lời khuyên tinh thần suông — nhiều người giải bài chuyên nghiệp nói về hiện tượng lời giải xuất hiện khi đang đi bộ hoặc tắm. Điều kiện để nó xảy ra là bạn phải đã nạp đủ vào đầu ở giai đoạn trước; rời bài mà chưa hiểu đề thì không có gì để áp cả.

Và một điều cuối, nói thẳng vì nó bị hiểu sai nhiều: *xem lời giải không phải điều xấu, xem lời giải quá sớm mới xấu.* Ngưỡng hợp lý khi luyện là 30–60 phút bí thật sự — bí sau khi đã chạy hết danh sách câu hỏi, chứ không phải bí sau khi đọc đề năm phút. Và khi đã xem, quy tắc bắt buộc là đóng lời giải lại rồi tự cài từ đầu, sau đó ghi vào nhật ký lỗi: *ý tưởng nào tôi đã không nghĩ ra, và dấu hiệu nào trên đề lẽ ra đã gợi ra nó.* Chương E/28 biến việc này thành một quy trình cụ thể.

1.6 Gốc rễ: quy trình này từ đâu ra

Bốn bước của Pólya trông như một danh sách hiển nhiên, nhưng nó là kết tinh của một truyền thống dài, và biết gốc rễ giúp bạn dùng nó đúng chỗ hơn.

Ý tưởng “có phương pháp để tìm ra lời giải, không chỉ để trình bày lời giải” xuất hiện từ Descartes trong *Quy tắc hướng dẫn tư duy* thế kỷ 17, và chính Pólya nhắc tới Descartes cùng Leibniz như những người đi trước. Điều thúc ép Pólya viết *How to Solve It* vào năm 1945 là một quan sát về dạy học: toán học được dạy như một *sản phẩm đã hoàn thành* — định lý, chứng minh, xong — trong khi quá trình thật sự tạo ra nó, gồm mò mẫm, thử ví dụ, đoán sai rồi sửa, thì bị giấu đi hoàn toàn. Người học vì thế tưởng rằng người giỏi “nhìn phát ra ngay”, và khi bản thân không nhìn ra ngay thì kết luận mình không có năng khiếu. Pólya viết sách để phơi bày phần bị giấu đó[1].

Đến thập niên 1970, ngành khoa học máy tính tiếp cận cùng câu hỏi từ hướng khác. Allen Newell và Herbert Simon nghiên cứu quá trình giải bài của con người bằng cách yêu cầu người ta *nói to suy nghĩ* khi giải, rồi mô hình hoá thành chương trình. Kết quả là khái niệm **không gian bài toán** (*problem space*) — giải bài tức là tìm đường trong một đồ thị các trạng thái — và **phân tích phương tiện–mục đích** (*means-ends analysis*): đo khoảng cách giữa trạng thái hiện tại và đích, rồi chọn hành động làm khoảng cách giảm. Đây là chỗ nổi rất đáng chú ý: cái mà ta gọi là “heuristic để nghĩ” trong chương này, và cái mà ta gọi là “hàm heuristic $h(n)$ ” trong thuật toán tìm kiếm A^* ở chương C/17, là *cùng một khái niệm*, ra đời từ cùng một dòng nghiên cứu. Tư duy của người và tìm kiếm của máy được mô hình hoá bằng cùng một ngôn ngữ.

Nhánh thứ ba là tâm lý học về sáng tạo. Graham Wallas mô tả bốn giai đoạn — chuẩn bị, ấp ủ, loé sáng, kiểm chứng — và giai đoạn “áp ủ” chính là cơ sở cho lời khuyên “rời bài ra” ở mục trước, kèm điều kiện quan trọng: phải nạp đủ ở giai đoạn chuẩn bị thì mới có gì để áp. Cuối cùng, Schoenfeld làm việc mà Pólya không làm được vì thiếu công cụ: đo thực nghiệm quá trình giải, và phát hiện ra rằng heuristic thôi chưa đủ, còn phải có tầng tự giám sát[2].

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thế kỷ 17 — Descartes, Leibniz	Tìm một “phương pháp phổ quát” để dẫn dắt tư duy, thay vì mỗi bài mỗi mẹo.	Đặt ra tham vọng có phương pháp; nhưng quá tổng quát nên không dùng trực tiếp được.
1945 — Pólya	Toán được dạy như sản phẩm hoàn chỉnh; người học không thấy quá trình mò mẫm nên tưởng mình thiếu năng khiếu.	Bốn bước và hệ thống câu hỏi; lần đầu quá trình giải bài trở thành thứ dạy được.
1972 — Newell & Simon	Muốn mô hình hoá tư duy đủ chính xác để viết thành chương trình.	Không gian bài toán và phân tích phương tiện–mục đích; heuristic trở thành khái niệm hình thức, dùng chung cho cả người lẫn máy ($\rightarrow A^*, C/17$).
1985 — Schoenfeld	Dạy heuristic Pólya cho sinh viên mà họ vẫn không giải được bài mới.	Phát hiện tăng tự giám sát; giải thích vì sao “biết heuristic” không đủ.
1993 — Ericsson	Vì sao cùng số giờ luyện mà người tiến người không.	Bốn điều kiện của luyện tập có chủ đích; nối chương này với chương A/02.

1.7 Liên hệ ngang: cùng một quy trình ở những nơi khác

Điều đáng để ý là quy trình bốn bước không phải đặc sản của toán hay của lập trình. Nó xuất hiện gần như y hệt ở mọi nghề mà công việc chính là *chẩn đoán trong điều kiện thiếu thông tin*, và nhìn thấy sự trùng khớp đó giúp bạn mượn được kỹ thuật từ nghề khác.

Ví dụ rõ nhất là gỡ lỗi phần mềm. Khi một chương trình sai, người có kinh nghiệm không đọc code từ đầu tới cuối; họ *chia đôi vùng nghi ngờ*: đặt một điểm kiểm tra ở giữa, xem trạng thái ở đó đúng hay sai, và loại bỏ một nửa. Đó chính là tìm kiếm nhị phân, áp lên không gian nguyên nhân thay vì lên mảng — và chương E/25 sẽ chỉ ra rằng git bisect là đúng thuật toán đó, đóng gói thành công cụ.

Ví dụ thứ hai đến từ chính công việc của người viết dòng này: gỡ một hệ thống SLAM chạy sai. Quy trình hiệu quả không phải đọc lại toàn bộ mã ước lượng, mà là: dựng *ví dụ nhỏ nhất* (một đoạn dữ liệu vài giây thay vì cả chuỗi), tìm một *bất biến* phải luôn đúng (hướng trọng lực không được trôi, hiệp phương sai phải xác định dương), rồi tìm bước đầu tiên mà bất biến bị vi phạm. Ba từ khoá — ví dụ nhỏ nhất, bất biến, bước hỏng đầu tiên — là đúng ba công cụ của chương này và chương A/04, chỉ đổi miền áp dụng.

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Khoa học thực nghiệm	Giả thuyết $\rightarrow$ thí nghiệm có khả năng bác bỏ $\rightarrow$ sửa giả thuyết	Giống: đều tiến bằng cách loại trừ, và một phản ví dụ mạnh hơn nhiều ví dụ ủng hộ. Khác: nhà khoa học không bao giờ có “đáp án đúng” để đối chiếu, còn bạn thì có — nên bạn được phép dùng vết cặn làm trọng tài.
Chẩn đoán y khoa	Chẩn đoán phân biệt: liệt kê mọi khả năng rồi loại bằng xét nghiệm rẻ nhất trước	Giống hệ heuristic “bỏ bớt ràng buộc” và “trường hợp cực đoan”. Khác: chi phí sai lệch bất đối xứng — bỏ sót bệnh nặng đắt hơn nhiều so với dương tính giả, nên thứ tự xét nghiệm không tối ưu theo thông tin thuần túy.
Gỡ lỗi phần mềm	Chia đôi vùng nghi ngờ, git bisect	Cùng thuật toán với tìm kiếm nhị phân, áp lên không gian nguyên nhân. Điều kiện áp dụng cũng giống: cần tính đơn điệu — trước điểm đó thì đúng, sau điểm đó thì sai (E/25).

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Ước lượng trạng thái, SLAM	Ví dụ nhỏ nhất + bất biến (trọng lực, hiệp phương sai) + tìm bước hỏng đầu tiên	Cùng bộ ba công cụ với chương này và A/04. Khác: hệ thực có nhiều, nên “bất biến” trở thành “nằm trong khoảng tin cậy”, và phải phân biệt sai số hợp lệ với lỗi thật.
Trí tuệ nhân tạo cổ điển	Không gian trạng thái, hàm heuristic $h(n)$, A^*	Không phải chỉ giống — cùng một khái niệm, cùng nguồn gốc từ Newell & Simon. Bạn dùng heuristic để cắt bớt nhánh trong đầu, A^* dùng nó để cắt bớt nhánh trong hàng đợi ưu tiên (C/17).
Cờ vua, cờ vây	Tính toán biến (calculation) xen kẽ với đánh giá thể (evaluation)	Giống tăng tự giám sát của Schoenfeld: kỳ thủ mạnh không tính sâu hơn nhiều, họ biết <i>khi nào nên ngừng tính một biến</i> . Xem thêm A/02.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì kiến thức chỉ là một trong bốn thành phần của năng lực giải bài. Người giải được bài mới có thêm ba thứ: một quy trình để chạy khi chưa biết làm gì, một bộ heuristic để đổi cách nhìn bài toán, và quan trọng nhất là khả năng tự giám sát — biết mình đang đi hướng nào, đi bao lâu rồi, và khi nào phải bỏ. Schoenfeld chỉ ra thành phần thứ ba tạo ra khác biệt lớn nhất trong dữ liệu thực nghiệm của ông.
- A2. Vì bí không phải trạng thái thiếu sức nghĩ, mà là trạng thái lặp lại một biểu diễn. “Nghĩ kỹ hơn” giữ nguyên biểu diễn nên cho ra đúng kết luận cũ. “Vẽ ví dụ $n = 4$ ” thì đổi biểu diễn từ ký hiệu trừu tượng sang hình ảnh cụ thể, và cấu trúc thường chỉ lộ ra ở biểu diễn mới.
- A3. Vì vết cặn cho ba thứ mà bạn không có nếu bỏ qua nó: một phát biểu chính xác về cái đang tìm, một lời giải đúng để đối chiếu khi stress test, và quan trọng nhất là một câu hỏi tốt hơn để suy nghĩ — “vì sao nó chậm” thay cho “giải thể nào”. Hình 1.1 cho thấy câu hỏi đó chỉ có vài câu trả lời khả dĩ, mỗi câu dẫn thẳng tới một nhóm kỹ thuật.
- A4. Bạn mất khả năng dừng lại. Bước 4 là chỗ ý tưởng được tách khỏi đề bài cụ thể và phát biểu thành một câu tổng quát — “cố định điểm cuối, duy trì đáp án tốt nhất kết thúc tại đó”. Không có bước 4, bạn tích lũy được những lần giải bài chứ không tích lũy được công cụ, nên bài sau vẫn phải nghĩ lại từ đầu.
- A5. Khi hướng đó đã chạy đủ lâu mà không tạo ra *tiến bộ kiểm chứng được* — không có bỏ đề mới, không có ví dụ nhỏ nào được giải, không có ràng buộc nào bị loại. Cách biết trước khi hết giờ là đặt mốc kiểm tra định kỳ, ví dụ 10 phút một lần, và trả lời ba câu ở mục trước. Không có mốc kiểm tra thì bạn chỉ phát hiện ra hướng sai khi đồng hồ đã hết.

NÓI LẠI THẬT ĐƠN GIẢN

Giải một bài lạ giống như tìm đường trong một thành phố lạ mà không có bản đồ. Việc đầu tiên không phải là chạy nhanh, mà là đứng lại xem mình đang ở đâu và cần tới đâu — đó là bước hiểu đề. Việc thứ hai là tìm một con đường bạn từng đi qua có nối tới đó không — đó là bước tìm mối nối, và mẹo hiệu quả nhất là đi bộ thử một đoạn ngắn bằng cách làm tay ví dụ nhỏ. Cách chậm nhưng chắc chắn tới nơi là đi thử hết mọi ngõ, tức vết cặn; bạn luôn nên hình dung cách đó trước, vì khi biết nó chậm ở chỗ nào thì bạn biết cần rẽ tắt ở đâu. Khi đã tới nơi rồi, hãy dành một phút nhìn lại con đường và ghi nhớ khúc quan trọng nhất — lần sau bạn sẽ không phải dò lại. Còn khi lạc, đừng đi nhanh hơn theo hướng cũ; hãy dừng lại và tự hỏi một câu hỏi khác.

Một câu: Khi bí, đừng nghĩ mạnh hơn — hãy đổi câu hỏi mà bạn đang tự hỏi.

Chỗ không rút gọn được: Không có cách nào biết chắc một hướng là sai trước khi thử; mọi thứ ta làm được chỉ là đặt mốc kiểm tra để phát hiện sớm và trả giá rẻ hơn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Bốn bước của Pólya	Hiểu bài toán → lập kế hoạch → thực hiện → nhìn lại. Không phải đường thẳng mà là vòng lặp: mắc ở bước sau thì quay lại bước ngay trước nó.
●●● Kiểm tra “đã hiểu đề”	Phát biểu lại đề bằng lời của mình, không nhìn vào đề. Chưa làm được thì mọi phút suy nghĩ tiếp theo đều phí.
●●● Vì sao luôn viết vết cạ trước	Nó cho phát biểu chính xác bài toán, cho lời giải đúng để stress test, và đổi câu hỏi từ “giải thế nào” thành “vì sao chậm” — câu sau dễ trả lời hơn nhiều.
●●● Ba lý do vết cạ chậm	Tính lại thứ đã tính (→ quy hoạch động); duyệt nhánh chắc chắn tẻ (→ tham lam, cắt tỉa); nhìn sai dạng bài (→ mô hình hoá lại).
●●● Tầng “control” của Schoenfeld	Khả năng tự giám sát: đang làm gì, có tiến bộ không, khi nào bỏ hướng. Đây là khác biệt lớn nhất giữa người giải giỏi và kém, và là thành phần dễ luyện nhất.
●● Heuristic hiệu quả nhất	Làm tay ví dụ nhỏ $n = 1, 2, 3, 4$ và xếp kết quả thành bảng. Chi phí thấp nhất, tỉ lệ ra ý tưởng cao nhất.
●● Heuristic “đảo ngược”	Đếm cái xấu rồi trừ; đi từ cuối về đầu; đổi “đáp án là bao nhiêu” thành “đáp án có $\geq x$ không” — câu cuối là cửa vào nhị phân trên đáp án.
●● Heuristic “bỏ bớt ràng buộc”	Giải bài dễ hơn trước (mọi số dương, đồ thị là cây, n nhỏ), rồi tìm cách bù lại ràng buộc đã bỏ.
●● Câu hỏi “cái gì đổi giữa hai bước”	Nếu đáp án bước $i + 1$ chỉ khác bước i một chút thì có lời giải tăng dần: hai con trỏ, cửa sổ trượt, cấu trúc dữ liệu động.
●● Quy tắc trước khi gõ code	Nói được lời giải bằng ba câu. Không nói được nghĩa là kế hoạch chưa xong, và gõ code chỉ chuyển sự mơ hồ vào trình soạn thảo.
●● Nội dung bước 4	Có ngắn/nhanh/đơn giản hơn không; ý tưởng cốt lõi phát biểu một câu không nhắc đề bài; tôi bí ở đâu và dấu hiệu nào lẽ ra giúp tôi thoát sớm.
●● Ba cách thoát khi hết heuristic	Giải một bài dễ hơn; viết ra chính xác chỗ kẹt dạng “tôi biết làm nếu... nhưng không biết khi...”; rời bài ra sau khi đã nạp đủ.
● Ngưỡng xem lời giải	30–60 phút bí thật sự, sau khi đã chạy hết danh sách câu hỏi. Xem xong bắt buộc đóng lại, tự cài từ đầu, và ghi nhật ký lỗi.
● Bốn thành phần của Schoenfeld	Kiến thức nền, heuristic, tự giám sát, và hệ niềm tin về việc giải toán là gì.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

- Vì sao Pólya nói “nếu bạn không giải được bài toán này, hãy tìm một bài dễ hơn liên quan tới nó”? Nghe như lời khuyên né tránh.

| Vì bài dễ hơn thường chứa *cùng cấu trúc* với bài gốc, chỉ bớt đi phần gây nhiễu. Khi giải được bài dễ, bạn có một lời giải cụ thể để soi, và câu hỏi trở thành “ràng buộc tôi đã bỏ đi làm hỏng chỗ nào của lời giải này” — một câu hỏi hẹp và cụ thể, dễ hơn hẳn câu hỏi mở ban đầu. Đây không phải né tránh mà là chia bài toán khó thành hai bước dễ hơn.

- Tại sao bảo “đọc giới hạn n là một nửa gợi ý”? Chẳng phải giới hạn chỉ là chi tiết kỹ thuật?

| Vì giới hạn loại bỏ phần lớn không gian tìm kiếm ý tưởng. Với $n \leq 20$, người ra đề gần như chắc chắn muốn một lời giải mũ, thường là duyệt là duyệt 2^n tập con hoặc quy hoạch động trên bitmask — và bạn không cần phí thời gian tìm lời giải đa thức thông minh. Với $n \leq 2 \cdot 10^5$, lời giải phải cỡ $O(n)$ hoặc $O(n \log n)$, nên mọi ý tưởng $O(n^2)$ bị loại ngay và bạn tiết kiệm được nửa giờ.

- Nếu tôi đã biết bài này là quy hoạch động, tôi có cần bước 1 và bước 2 không?

| Có, và bước 1 lúc này còn quan trọng hơn. Biết “là quy hoạch động” chưa nói được trạng thái là gì — mà chọn sai trạng thái là cách hỏng phổ biến nhất của quy hoạch động. Trạng thái đúng thường lộ ra từ việc làm tay ví dụ nhỏ và từ câu hỏi “cái gì thực sự thay đổi giữa hai bước liên tiếp”, tức là từ bước 1 và bước 2.

4. Trong một kỳ thi có giới hạn thời gian, bốn bước của Pólya có xa xỉ quá không?

| Ngược lại, dưới sức ép thời gian thì bốn bước lại càng đáng giá, chỉ là mỗi bước ngắn lại. Kinh nghiệm thi đấu cho thấy chi phí lớn nhất không phải nghĩ chậm mà là *cài xong rồi mới phát hiện hiểu sai đề*, hoặc *đi một hướng sai 40 phút*. Bước 1 rút gọn còn hai phút và bước “dừng lại tự đánh giá” chỉ tốn 20 giây, nhưng chúng chặn đúng hai lỗi tốn kém nhất đó.

5. Làm sao phân biệt “bí lành mạnh” (nên tiếp tục) với “bí vô ích” (nên xem lời giải)?

| Bằng dấu hiệu tiến bộ, không bằng cảm giác. Bí lành mạnh là khi mỗi 10–15 phút bạn vẫn thu được thứ mới: một ví dụ nhỏ giải được, một hướng bị loại có lý do, một bất biến được phát hiện. Bí vô ích là khi bạn đọc lại đề lần thứ năm và nghĩ đúng những ý đã nghĩ. Dấu hiệu thứ hai xuất hiện thì hoặc đổi heuristic, hoặc chuyển bài, hoặc xem gợi ý — gợi ý một dòng, không phải toàn bộ lời giải.

6. Vì sao chương này nhấn mạnh “phát biểu ý tưởng bằng một câu không nhắc tới đề bài”?

| Vì đó là cách kiểm tra xem bạn đã trừu tượng hoá được chưa. Câu nói kèm chi tiết đề bài (“với mảng số trong bài này, ta duyệt từ trái sang”) chỉ dùng được cho đúng bài đó. Câu nói tổng quát (“khi tìm cực trị trên mọi đoạn, cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó”) dùng được cho hàng chục bài khác. Kiến thức chỉ dùng lại được ở dạng thứ hai.

7. Người ta hay nói “giải nhiều bài thì tự khắc giỏi”. Chương này có phản đối điều đó không?

| Có, một phần. Số lượng chỉ có tác dụng nếu mỗi bài đều đi qua bước 4 và nếu độ khó được chọn hơi cao hơn mức hiện tại của bạn — đúng định nghĩa luyện tập có chủ đích của Ericsson[3]. Giải một trăm bài vừa sức, không nhìn lại, không ghi lỗi, thì bạn luyện tốc độ gõ chứ không luyện năng lực giải bài. Chương E/28 biến nhận xét này thành một lịch luyện cụ thể.

8. Bất biến xuất hiện ở đây làm gì, chẳng phải nó thuộc về phần chứng minh?

| Bất biến có hai vai trò, và vai trò sinh ý tưởng thường bị bỏ quên. Khi bạn hỏi “đại lượng nào không đổi trong quá trình này”, bạn vừa đang tìm công cụ chứng minh, vừa đang tìm chính cấu trúc để thiết kế thuật toán — ví dụ bất biến “cur là tổng lớn nhất của đoạn kết thúc tại j ” vừa là lời giải vừa là chứng minh của bài tổng đoạn lớn nhất. Chương A/04 khai thác cả hai vai trò.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tổng đoạn con lớn nhất	CSES “Maximum Subarray Sum”	Đi lại đúng bốn bước của mục 4; viết bất biến ra thành lời trước khi gõ code.
● Đảo ngược một số nguyên, xử lý tràn	LeetCode “Reverse Integer”	Bước 1 quyết định tất cả: liệt kê trường hợp biên (số âm, tận cùng bằng 0, tràn 32 bit) trước khi viết.
● Hai số có tổng bằng x	CSES “Sum of Two Values”	Heuristic “sắp xếp lại dữ liệu” hoặc “đổi sang tra cứu”; so sánh hai lời giải và giải thích khi nào chọn cái nào.
●● Bài toán tám quân hậu	CSES “Chessboard and Queens”	Vết cạn có cắt tỉa; luyện thói quen ước lượng kích thước không gian tìm kiếm <i>trước</i> khi cài.
●● Tìm phần tử xuất hiện quá nửa	LeetCode “Majority Element”	Heuristic “đại lượng bất biến”: thuật toán bỏ phiếu Boyer–Moore. Hãy tự tìm bất biến trước khi tra.
●● Tích đoạn con lớn nhất	LeetCode “Maximum Product Subarray”	Cùng mẫu với bài mở đầu nhưng số âm phá cấu trúc: cần giữ đồng thời cực đại và cực tiểu. Ví dụ nhỏ lộ ra ngay.
●● Trò chơi Nim đơn giản	Bài tập kinh điển	Heuristic “bất biến” và “trường hợp cực đoan”: làm tay các thể $n \leq 3$ trước khi tìm quy luật XOR.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Số cách leo cầu thang với bước 1 và 2	AtCoder DP “Frog 1”	Bài mẫu để tập bước 4: phát biểu ý tưởng bằng một câu không nhắc tới cầu thang.
●●● Bài toán 3-SUM	Bài tập kinh điển	Từ $O(n^3)$ xuống $O(n^2)$: cố định một biến rồi dùng hai con trỏ. Luyện heuristic “giá mà tôi biết trước một đại lượng”.
●●● Xếp lịch cuộc họp tối đa	Bài tập kinh điển	Bí thì hỏi “trường hợp cực đoan nói gì”: chọn theo thời điểm kết thúc sớm nhất. Thử tự chứng minh bằng lập luận đối chỗ (chương A/04).
●●● Bài toán “nước mưa mắc kẹt”	LeetCode “Trapping Rain Water”	Ba lời giải khác nhau (tiền tố cực đại, ngăn xếp, hai con trỏ). Giải cả ba rồi so, đây là bài tập bước 4 tốt nhất trong danh sách này.
●●● Một bài bạn từng bí và đã xem lời giải	Nhật ký của chính bạn	Tự cài lại không nhìn, rồi viết một câu về ý tưởng cốt lõi và một câu về dấu hiệu lẽ ra đã gợi ra nó.

Kết chương và đọc tiếp

Chương này cố ý không chứa thuật toán nào. Nó dựng một quy trình bốn bước, một danh sách mười hai câu hỏi để phá thế bí, và một thói quen tự giám sát bằng mốc thời gian. Ba thứ đó là khung để treo mọi kiến thức kỹ thuật của 27 chương còn lại; thiếu chúng, kiến thức chỉ nằm thành danh mục và không tự tìm được đường ra khi cần.

Còn bỏ ngỏ một câu hỏi lớn: quy trình thì có rồi, nhưng *luyện* thế nào để nó thành phản xạ, và những người giỏi nhất thực sự luyện ra sao? Đó là nội dung chương A/02. Sau đó, chương A/03 và A/04 cấp hai cái thước — nhanh không, đúng không — mà bước 2 và bước 3 của Pólya luôn cần tới.

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, "The Role of Deliberate Practice in the Acquisition of Expert Performance," *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.